

Large Language Models for Software Requirements Classification: Cross-Dataset Generalization and Cost-Efficiency

Fatma El-Zahraa Samir, Khaled T. Wassif, and Lamia AbouZeid
Faculty of Computers and Artificial Intelligence, Cairo University, Giza, Egypt
fatmaelzahraasamirabdelfattah@gmail.com

Abstract—Several recent studies report that prompted large language models (LLMs) match or exceed fine-tuned transformer baselines on software requirements classification, and some of them conclude that labelled training data is no longer needed for the task. Those results are almost always obtained in-distribution, with training data or few-shot exemplars drawn from the same corpus that supplies the test items, and they rarely report what the technique costs to run. This paper re-examines the comparison after repairing both omissions. Fine-tuned encoders and prompted LLMs of three access tiers classify the same requirements from two public corpora, under the same tasks and the same scoring rule, at three levels of distribution shift: within one corpus, across held-out projects, and across an independently annotated corpus. Every encoder–LLM comparison is paired on identical items and corrected for multiplicity as one test family, and inference cost is measured per model from the same runs that produced the predictions. The ranking of the two approaches is not stable across those levels. Within a corpus the fine-tuned encoders are ahead and are never significantly beaten; once the test corpus is annotated by someone else the ordering reverses and the prompted models are ahead in most comparisons. The reason is not that the transferred encoder fails to recognise the vocabulary of the new corpus but that it carries the class balance of the corpus it was trained on, and the prompted models turn out to have the same weakness for a different reason: their class balance is set by the wording of the instruction, and changing that wording moves it. Encoders remain an order of magnitude cheaper per item and recover their one-off training cost within a few hundred classifications. The practical implication is that the evaluation regime belongs in the claim: a result obtained in-distribution does not predict the ranking under a change of corpus, and can reverse it.

Index Terms—requirements classification, large language models, fine-tuning, generalisation, distribution shift, empirical software engineering, cost-efficiency

I. INTRODUCTION

Requirements classification assigns natural-language requirement statements to categories such as functional (FR) versus non-functional (NFR), or security-relevant versus not. ISO/IEC/IEEE 29148 defines requirements and their attributes for the whole life cycle [2], and classification is one of the first analytical steps performed once requirements have been elicited. Getting it right supports early risk identification and architectural choice; getting it wrong propagates downstream as rework [1]. Two decades of automation have moved the task from weighted indicator terms [1], through supervised learning on engineered features [3], [5], to fine-tuned transformers [4]

and, since 2024, to prompted generative LLMs [7], [11]–[13], [16]. A systematic mapping of AI techniques applied to the task documents the same progression and the datasets it has relied on [8].

It is worth being precise about what this recent work claims. No study we are aware of states that requirements classification is solved. What several studies report is parity or better between prompting and supervision on the corpora they use: Binkhonain and Alfayaz find that few-shot prompting matches or exceeds a fine-tuned transformer and conclude that prompting is a viable alternative where labelled data is scarce [11]; Tang et al. report an 8 B open model passing a fine-tuned baseline on PROMISE [13]; and Alhoshan et al. report that generative models perform the task competitively without task-specific training [12], extending earlier evidence that zero-shot embedding methods already made labelled data non-essential in some settings [6]. Together these results make “prompting can replace fine-tuning” a reasonable reading of the state of the art, and it is that reading, rather than any individual paper’s claim, that this study tests.

The reading rests on a measurement convention. Almost every published result is obtained in-distribution: the model is fine-tuned, or its few-shot exemplars are selected, on the same corpus that then supplies the test items. Where prior work reports improved generalisability, it usually means transfer across *tasks* on the same corpora rather than to unseen projects or an independently produced dataset [11]—the property the field’s own transfer-learning baseline warned about, since NoRBERT was motivated by classifiers over-fitting project-specific wording [4]. A second omission compounds the first: inference cost, latency and model size are seldom quantified, although they decide whether a technique can be adopted without a commercial API budget.

This paper asks what happens to the comparison when both omissions are repaired. We hold the corpus, the items, the label sets and the scoring rule fixed, and vary only the distribution shift between what a model was fitted on and what it is scored on. Our contributions are:

- 1) **A paired, multiplicity-controlled benchmark** of twelve configurations—four fine-tuned encoder variants and eight prompted LLMs across local, hosted and commercial access tiers—over three tasks, two corpora and three evaluation regimes, scored once under one strict protocol,

with all encoder–LLM tests corrected as a single family (Sections III and IV).

- 2) **Evidence that the ranking depends on the regime.** Within a corpus the encoders are ahead and are never significantly beaten; across corpora the same comparisons reverse (Section IV-B).
- 3) **A diagnosis rather than a gap.** The collapse under corpus change is consistent with a shift in class balance rather than with unfamiliar vocabulary, and the prompted models have the same defect from another source: their predicted class balance is set by the wording of the instruction (Sections IV-B and IV-C).
- 4) **A measured accuracy–cost comparison,** with the price of each configuration derived from the same runs that produced its predictions, and the compute-priced and token-priced tiers reported separately (Section IV-D).

Three research questions organise the study. **RQ1:** how do open and commercial LLMs compare with fine-tuned encoders when both are evaluated within one corpus? **RQ2:** how much performance is lost under cross-project and cross-dataset evaluation, and does the loss differ between the two model families? **RQ3:** what is the accuracy-versus-cost trade-off, and what is the cheapest freely available option that is good enough?

II. RELATED WORK AND THE EVALUATION GAP

A. From indicator terms to prompting

Automated requirements classification has passed through four overlapping phases. Rule-based work established the task and produced the PROMISE NFR collection [1]. Classical supervised learning followed, with SVMs separating FRs from NFRs [3] and interpretable variants combining dependency parsing with feature attribution [5]; deep models removed much of the feature engineering but needed more data [9]. Transfer learning then reset the state of the art: NoBERT fine-tunes BERT and reports macro-F1 in the mid nineties on the PROMISE categories [4], a pattern the systematic mapping of Kaur and Kaur later confirmed across the published literature [8]. Zero-shot sentence-embedding methods then showed that labelled data was not strictly necessary [6], and prompted generative LLMs now dominate recent work.

B. Recent LLM studies (2024–2026)

El-Hajjaji et al. [7] compared SVM and LSTM against GPT-3.5 and GPT-4 over five datasets and found no single best technique; Peer et al. [10] embedded LLM classification in a model-based systems engineering workflow. Binkhonain and Alfayaz [11] report the study closest to ours: five LLMs under four prompting strategies over three tasks on PROMISE and SecReq, benchmarked against a fine-tuned transformer that few-shot prompting matches or exceeds. Alhoshan et al. [12] ran more than 400 experiments over three open generative models, and Tang et al. [13] identified an over-prompting effect in which additional in-context examples degrade accuracy on PROMISE and PURE [19], while a TF–IDF exemplar selector pushes an 8B open model past the fine-tuned baseline. Chaudhary et

TABLE I
EVALUATION COVERAGE OF RECENT LLM-BASED REQUIREMENTS-CLASSIFICATION STUDIES. ✓ = REPORTED; ✗ = NOT REPORTED; ~ = PARTIALLY REPORTED. “CROSS-PROJECT” MEANS HELD-OUT UNSEEN PROJECTS; “CROSS-DATASET” MEANS AN INDEPENDENTLY PRODUCED CORPUS.

Study	Open LLMs	Comm. LLMs	Cross-project	Cross-dataset	Cost
El-Hajjaji et al. [7]	✗	✓	✗	✗	✗
Peer et al. [10]	✗	✓	✗	✗	~
Binkhonain & Alfayaz [11]	✓	✓	✗	✗	✗
Alhoshan et al. [12]	✓	✗	✗	✗	✗
Tang et al. [13]	✓	✓	✗	✗	✗
Chaudhary et al. [14]	✗	✓	✗	~	✗
Vasudevan et al. [15]	✗	✓	✗	✗	✗
Levy et al. [16]	✓	✓	✗	✗	✗

al. [14] extended the task to noisy app-store reviews, Vasudevan et al. [15] compared fine-tuned BERT and RoBERTa against GPT-4o, and Levy et al. [16] benchmarked three current models against expert judgment on PROMISE_exp.

C. What this literature does not measure

Table I summarises the evaluation coverage of these studies along the axes that matter for adoption, and three patterns recur. *First*, evaluation is in-distribution: the same corpus supplies the training data or the few-shot exemplars and the test items. Only the app-review study [14] tests genuinely different text, and it reports markedly lower precision on several categories—a signal the literature has not followed up. *Second*, cost is not a reported dimension, even in the studies that use open models [12], [13]. *Third*, the same two or three small, ageing, public corpora underlie nearly every result, so contamination of LLM pre-training data is a threat that is acknowledged but not addressed [34].

These gaps interact. If in-distribution parity is what motivates replacing supervision with prompting, and is also what corpus reuse and contamination would produce spuriously, then the comparison that would settle the question has not been run. This study runs it: the same models, items and scoring rule, evaluated at three levels of distribution shift, with inference cost measured alongside accuracy.

III. STUDY DESIGN

The study is a controlled benchmark. Twelve model configurations classify the same requirements, under the same tasks, against the same frozen splits, scored by one rule, so every comparison we report is paired on identical items. Fig. 1 shows the design: one data and scoring path shared by both model families, and three evaluation regimes of increasing distribution shift. The pipeline runs as seven stages, each writing a content-hashed artefact that the next consumes; every number in Section IV is read back from those artefacts by the scripts that typeset the tables and figures.

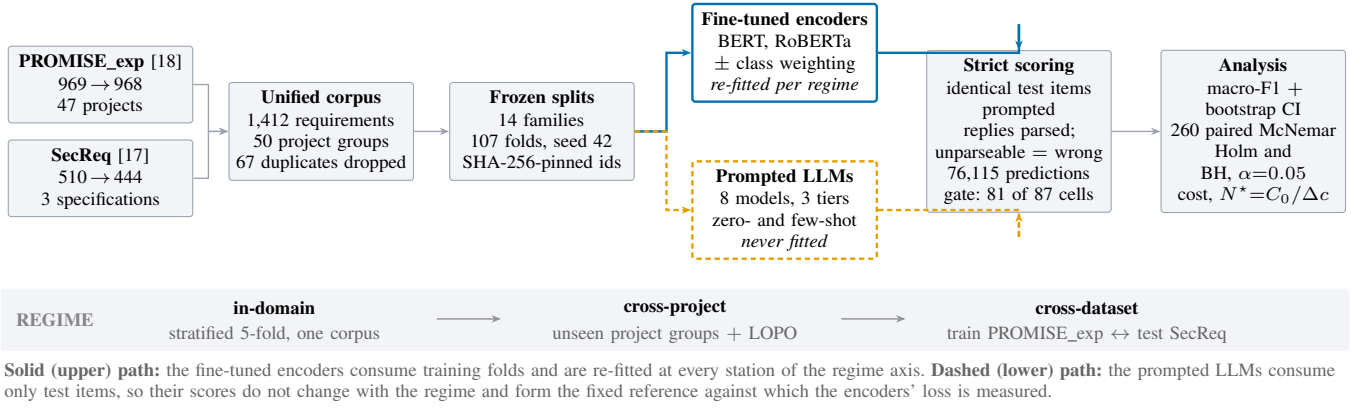


Fig. 1. Experimental architecture. One data path (left) feeds two model families that differ in exactly one respect: whether they consume training data. Both answer the *same* items, so every encoder–LLM comparison is paired item-for-item, and both pass through one strict scoring path (right). The band beneath is the study’s independent variable: the distribution shift between what a model was fitted on and what it is scored on.

A. Data preparation

Two public corpora are used. PROMISE_exp [18], an expanded release of the PROMISE NFR collection, contributes 969 requirements from 47 projects, each labelled FR or NFR and, when NFR, with one of eleven quality sub-types. SecReq [17] contributes 510 sentences from three industrial security specifications, each labelled security-relevant or not.

Preparation proceeds in four steps, implemented once in the first pipeline stage. (i) *Normalisation*: texts are whitespace- and encoding-normalised, and label strings are mapped onto one controlled vocabulary so that the two corpora express the security concept identically. (ii) *De-duplication*: exact duplicate texts are removed—67 rows, all of them within a corpus rather than across the two—leaving 1,412 requirements in 50 project groups (Table II). (iii) *Split generation*: the fourteen split families used by the three regimes are generated once with seed 42 and stored, keyed by requirement identifiers pinned to the SHA-256 hash of each source file. Because grouped-fold membership depends on the library version that produced it [23], the split file is consumed by later stages and never regenerated. (iv) *Frame construction*: each task defines the set of items it is evaluated on, as described below.

Two properties of the result matter for the rest of the paper. The first is a difference in class balance: security requirements are 12.9% of PROMISE_exp but 39.9% of SecReq, a roughly threefold difference in prevalence for a concept both corpora annotate independently, and the cross-dataset regime is built on that contrast. The second is that the security label reaches the two corpora by different routes. In PROMISE_exp it is derived from the security sub-class of the NFR taxonomy, so every PROMISE_exp item labelled security is also labelled NFR and none of the 444 FRs is labelled security; in SecReq it is applied to any security-relevant sentence, with no FR/NFR annotation at all. The cross-dataset regime therefore carries a definitional shift alongside the distributional one, and we return to this in the threats to validity.

TABLE II

CORPUS CONSTRUCTION. “SEC. PREV.” IS THE SHARE OF ITEMS LABELLED *security*; THE THREEFOLD DIFFERENCE BETWEEN THE TWO CORPORA IS WHAT THE CROSS-DATASET REGIME EXPLOITS. COUNTS ARE READ FROM THE STORED CORPUS FILE, NOT TRANSCRIBED FROM THE SOURCE PAPERS.

Corpus	Raw	Final	Groups	FR/NFR	Sec. prev.
PROMISE_exp [18]	969	968	47	444 / 524	12.9%
SecReq [17]	510	444	3	—	39.9%
Unified	1,479	1,412	50	444 / 524	21.4%

B. Tasks, label sets, and why they are not staged

Three task families are defined over the unified corpus. *Binary FR/NFR* uses the native PROMISE_exp labels over all 968 PROMISE_exp items. *Binary security* is native in SecReq and derived in PROMISE_exp from the security sub-class, which gives the same task on two independently produced corpora. *NFR sub-type* classification uses the eleven-class taxonomy of Cleland-Huang et al. [1], whose categories align with the ISO/IEC 25010 quality characteristics [24].

The three tasks are evaluated independently. There is no cascade: the security classifier is not shown the output of the FR/NFR classifier, and every item is presented to it directly, including the 444 PROMISE_exp items labelled FR. We state this because the PROMISE_exp *annotation* is hierarchical—security is a sub-class of NFR—so it would be easy to assume the *pipeline* is hierarchical too. It is not, and deliberately so: a staged design would make the security score depend on the FR/NFR classifier’s errors, and would have no counterpart in SecReq, where no FR/NFR labels exist. Keeping the tasks flat is what makes the security task comparable across the two corpora.

Several sub-type classes have single-digit support in PROMISE_exp, so the sub-type task is evaluated at three granularities: the full eleven classes (524 items), the six most frequent (433) and the four most frequent (353). For a fine-tuned encoder the label set determines the size of the classification head and the model is re-trained for each

granularity. For a prompted model there is no head to resize and no training at all, so it is fair to ask what top-4 and top-6 mean. They change three things. The candidate labels are enumerated inside the prompt itself, so at top-4 the model is offered four labels and at all-11 eleven. The evaluation frame changes: an item enters the top-4 frame only if its true label is one of those four. And macro-F1 is computed over the full declared label set of the variant, so a model is penalised for every declared class it never predicts, and that denominator grows with granularity. Granularity is a property of the question and of the scoring rather than of the model weights, and it is therefore measurable for prompted and fine-tuned models alike.

C. Evaluation regimes

Fourteen frozen split families, comprising 107 folds, implement three regimes of increasing distribution shift. **In-domain** is stratified 5-fold cross-validation within one corpus. This is the in-distribution setting that prior work reports. **Cross-project** uses grouped folds that hold out entire unseen project groups, plus a leave-one-project-out variant over the 47 PROMISE_exp projects. **Cross-dataset** fits on the security task of one corpus and tests on the other, in both directions: identical label sets, different annotation provenance.

The asymmetry between the two model families is the analytical lever. Encoders are re-fitted at every regime on that regime’s training folds; prompted models have no training distribution, so their scores cannot change with the regime, which is why the same score appears in every regime column of Table VI. Every requirement is a test item exactly once within each encoder family, so the encoders’ stored predictions cover the corpus. The prompted models answer stratified samples of the same items—300 per binary cell and 200 per sub-type cell, allocated in proportion to the class prior with a floor of ten per class—and the six local models additionally answer the full evaluation frames. Every encoder–LLM comparison is paired on identical requirements, aligned by identifier rather than by position.

D. The two model families, and why they are compared

Table III lists the twelve configurations. The comparison is between two architectures, and the difference is not incidental to the question.

A fine-tuned encoder is a bidirectional, encoder-only transformer pre-trained with masked language modelling and then adapted by supervised training of a classification head over the pooled sentence representation. Both the decision boundary and the class prior are learned from the labelled training folds. A prompted LLM is a decoder-only autoregressive model that has not been adapted to the task at all: it receives the label vocabulary as text, produces the answer as generated tokens, and its implicit class prior comes from pre-training and from the instruction rather than from any corpus of requirements.

That contrast is what makes the comparison informative, because the two families should respond differently to distribution shift precisely by acquiring their class prior from different places—and Section IV-C shows that they do. Prior

TABLE III
THE TWELVE MODEL CONFIGURATIONS. “B” IS THE PUBLISHED PARAMETER COUNT OF THE EXACT CHECKPOINT, IN BILLIONS; GEMINI’S IS UNDISCLOSED AND IS RECORDED AS UNKNOWN RATHER THAN ESTIMATED.

Config.	Checkpoint / endpoint	B	Access
BERT (w / u)	bert-base-uncased	0.11	fine-tuned
RoBERTa (w / u)	roberta-base	0.13	fine-tuned
Gemma-2-2B	gemma-2-2b-it	2.61	local, 4-bit
SmolLM3-3B	SmolLM3-3B	3.08	local, 4-bit
Qwen2.5-3B	Qwen2.5-3B-Instruct	3.09	local, 4-bit
Phi-4-mini	Phi-4-mini-instruct	3.84	local, 4-bit
Qwen2.5-7B	Qwen2.5-7B-Instruct	7.62	local, 4-bit
Llama-3.1-8B	Llama-3.1-8B-Instruct	8.03	local, 4-bit
Llama-3.3-70B	llama-3.3-70b-versatile	70.6	hosted
Gemini 3.1 F-Lite	gemini-3.1-flash-lite	n/a	commercial

work has argued that architectural choice affects requirements classification results [15], so we treat architecture as a declared property rather than a nuisance variable: Table III records the family, parameter count and access tier of every model, and all results are reported per configuration.

We fine-tune bert-base-uncased [20] and roberta-base [21] with a classification head (AdamW, $1r \times 10^{-5}$, batch 16, max length 128, linear decay without warmup, full precision; 3 epochs on the binary tasks, 10 on the sub-types), each with inverse-frequency class weighting in the loss, plus an unweighted control on the security task. The local tier runs six instruction-tuned models of 2.6–8.0 B parameters on a single free-tier NVIDIA T4 under 4-bit NF4 quantisation [22]. The hosted tier accesses Llama-3.3-70B and the commercial tier Gemini 3.1 Flash-Lite, both through free API tiers. A ninth prompted model was attempted and failed the reportability gate defined below; its predictions remain in the raw store but enter no table.

E. Prompt design and what it is meant to control

Each prompted model answers a fixed instruction that names the task, enumerates the label vocabulary and requires exactly one word in reply, sent as a single user message (Fig. 2). Decoding is greedy—temperature 0, at most twelve new tokens—so the only stochastic element is item sampling.

Fixing the instruction needs justifying, because a different wording could give a different result. Three points define what the choice does and does not claim.

Where the wording comes from. The template follows the zero-shot format used in the LLM requirements-classification studies we compare against [11]–[13]: a short task statement, an explicit list of permitted labels, and an instruction to answer with the label alone. We did not tune it against the test items. A conventional instruction keeps the comparison with prior work interpretable; a tuned one would not.

We do not claim it is the best prompt. Almost certainly it is not. What the design requires is a *fixed* instruction, so that the only quantity varying across the three regimes is the distribution shift. If the prompt were re-optimised per regime,

```

You are classifying a software requirement.
Decide whether it is a FUNCTIONAL requirement (FR) --
something the system must DO -- or a NON-FUNCTIONAL
requirement (NFR) -- a quality, constraint or property
such as performance, security or usability.
Answer with exactly one word: "FR" or "NFR".

<few-shot conditions only>
Examples:
Requirement: ""exemplar""
... k class-balanced exemplars, carved out before the evaluation frames
Now classify this one:
Requirement:
""the requirement under test""
Answer:

```

Fig. 2. The *base* FR/NFR instruction, verbatim, and where few-shot exemplars are injected. The sub-type tasks use the same template with the permitted labels listed in place of “FR”/“NFR”, which is how the top-4, top-6 and all-11 variants differ for a prompted model. The *terse* and *verbose* phrasings change only the instruction block.

TABLE IV
WHAT THE WORDING IS WORTH: MACRO-F1 UNDER EACH OF THE THREE WORDINGS ON THE SAME ITEMS, FOR THE NINE PAIRED CELLS OF THE PHRASING SUB-STUDY. “SPREAD” IS THE RANGE ACROSS THE THREE; “ORACLE” IS THE GAIN OVER *base* THAT A CHOOSER WITH ACCESS TO THE TARGET LABELS WOULD OBTAIN. PR = PROMISE_EXP, SR = SECREQ.

Model	Cell	base	terse	verb.	spread	oracle
Gemma-2-2B	FR/NFR/PR	0.671	0.665	0.688	0.022	+0.017
Gemma-2-2B	sec./PR	0.889	0.556	0.901	0.345	+0.012
Gemma-2-2B	sec./SR	0.827	0.343	0.828	0.485	+0.001
Qwen2.5-7B	FR/NFR/PR	0.675	0.678	0.670	0.008	+0.003
Qwen2.5-7B	sec./PR	0.879	0.819	0.872	0.059	+0.000
Qwen2.5-7B	sec./SR	0.851	0.629	0.848	0.222	+0.000
Qwen2.5-7B	sub-11/PR	0.606	0.591	0.626	0.036	+0.020
Qwen2.5-7B	sub-4/PR	0.889	0.868	0.862	0.027	+0.000
Qwen2.5-7B	sub-6/PR	0.813	0.849	0.805	0.044	+0.036

a difference between regimes could no longer be attributed to the shift. The fixed prompt is a control, not a recommendation.

What the wording is worth is measured, not assumed. Two sub-studies vary one factor at a time. The first re-runs two models over three phrasings of the same question—*base*, *terse* and *verbose*—with the answer contract held identical; Table IV reports all nine paired cells. Its last column gives the gain an oracle would obtain by picking the best of the three wordings for each cell: a median of +0.003 macro-F1 and a maximum of +0.036, far smaller than the differences this paper’s conclusions rest on. Picking that wording would in any case need labelled data from the target corpus, which is the resource prompting is meant to do without. The second sub-study is a few-shot ladder ($k \in \{2, 4, 8\}$ in total on the binary tasks, $k \in \{1, 2\}$ per class on the sub-types) with deterministic, class-balanced exemplars carved out of the corpus *before* the evaluation frames are formed, so an exemplar is never also a test item. The hosted and commercial tiers run zero-shot only, under provider rate limits.

F. Parsing, metrics, and how to read them

Outputs are parsed by a strict, word-boundary-aware rule: a leading label wins; otherwise exactly one label named anywhere in the reply wins; a reply naming both labels or neither is

unparseable and is scored as wrong. This is harsher than the lenient protocols common in prior work, and it matches deployment, where an unusable answer is not a free pass. The same rule is applied at generation time and again in an idempotent repair pass over the archived raw outputs, so stored and scored predictions cannot diverge.

Four kinds of quantity are reported, and because they are easy to confuse we state what each one is. *Macro-F1* is the unweighted mean of the per-class F1 scores over the full declared label set of the task variant; it is the primary metric because it does not let a model score well by ignoring a rare class, and the majority-class baseline in the last row of Table VI gives every score a floor to be read against. *Relative loss* ($\Delta\%$ in Table VII) is the share of a configuration’s own in-domain macro-F1 lost in a shifted regime, on the same items; it is defined only for the encoders, since only they have an in-domain fit to lose. *Predicted class share* is the fraction of test items a model assigns to a class. It is not an accuracy: it is what the model takes the class balance to be, read against the true prevalence in the same items. It and relative loss are both percentages, so Section IV-B names the class and the corpus each time one is used. *Test counts* are numbers of statistical comparisons, not of items: each unit is one exact McNemar test between one encoder configuration and one prompted model in one cell.

Uncertainty uses a stratified bootstrap [25]: 2,000 resamples drawn within each true class (seed 42), so no resample can lose a rare class. Encoder–LLM comparisons use the exact two-sided McNemar test on paired disagreements [26], [27]; all 260 are corrected as one family under both Holm [28] and Benjamini–Hochberg [29] at $\alpha = 0.05$, with verdicts keyed on the BH-adjusted p (175 significant before correction, 170 after BH, 131 after Holm). A cell—one model, task, dataset and regime—is reported only if every class has at least 5 test items, the cell holds at least 40 items, and the model parsed at least 50% of its responses; 6 of 87 cells fail this gate and are excluded rather than imputed. The store holds 76,115 predictions: 24,280 from the encoders, 35,049 prompted binary and 16,786 prompted sub-type.

G. Cost model

Every prediction row records input and output token counts, wall-clock latency and, for the encoders, training time. Cost is then computed from those recorded quantities rather than estimated afterwards, and the basis differs by tier because the two kinds of model are billed differently.

Models that run on our own hardware are priced on compute. That covers the fine-tuned encoders and all six local open-weight models: they are free to download and carry no licence fee, so the only thing to charge for is the machine time they consume. We price it as an imputed rental of the benchmark GPU—an AWS `g4dn.xlarge` with one NVIDIA T4, at the published on-demand list rate for `us-east-1` of \$0.5260 per hour, recorded in the pipeline manifest with the date it was checked. That is a list price for hardware, not a market price for the model, and it assumes sequential unbatched inference;

TABLE V

COST AND LATENCY BY ACCESS TIER, WITH THE BASIS ON WHICH EACH IS PRICED. USD/1k IS THE INFERENCE COST OF 1,000 CLASSIFICATIONS; WHERE A TIER HOLDS SEVERAL MODELS THE RANGE SPANS THEIR PER-MODEL MEDIANS. “TRAIN” IS THE ONE-OFF FINE-TUNING COST OF A DEPLOYABLE MODEL. RATES ARE THOSE OF SECTION III-G.

Tier	Mod.	B	Basis	USD/1k	Latency (s)	Train
Fine-tuned encoder	4	0.11–0.12	GPU time	0.0024	0.016	0.0093
Open-weight, local	6	2.61–8.03	GPU time	0.0312–0.0522	0.194–0.356	–
Open-weight, hosted	1	70.60	tokens	0.0704	0.095	–
Commercial API	1	n/a	tokens	0.0248	0.614	–

the stored sensitivity table reports the same costs at the spot rate (\$0.3162/h) and at batch sizes 8 and 32, which cut the local figures by roughly an order of magnitude.

Models behind an API are priced on tokens, at the provider’s published list rate per million input and output tokens—\$0.25/\$1.50 for Gemini 3.1 Flash-Lite and \$0.59/\$0.79 for Llama-3.3-70B, taken from the providers’ public rate cards in August 2026 and frozen into every prediction row at generation time, so a later price change cannot alter a stored result. Both models were in fact run on free tiers; the list rate is what the same volume would have cost.

Neither basis is a fixed property of the technique, and cost should be expected to move with model size and capability. Within the compute-priced tier it does: over the ten locally executed configurations, cost per item rises with parameter count (Spearman $\rho = 0.85$). That is why Table V puts parameter counts next to prices and why Section IV-D states the comparison as a ratio between tiers rather than an absolute price that would date. The ordering is not perfectly monotonic in size, because a locally run model is charged for the wall-clock time it occupies the GPU, and a model that emits longer replies pays for them at equal size.

Encoder cost separates the one-off fine-tuning cost C_0 from the marginal per-item cost c_{enc} , which gives a break-even volume against any prompted alternative

$$N^* = \frac{C_0}{c_{\text{alt}} - c_{\text{enc}}}. \quad (1)$$

Because free-tier latency is dominated by provider-side queuing, wall-clock projections use per-request median latency, with the queue-inflated mean reported separately.

IV. RESULTS

Table VI is the paper’s single results table. It carries all twelve configurations, the six task-by-corpus cells and every regime each cell supports, so encoders and prompted models are read off the same rows and columns. The encoders have one score per regime because they are re-fitted at each; a prompted model has one score per cell, written across the regime columns it spans, because a model that consumes no training data cannot score differently when the training data changes.

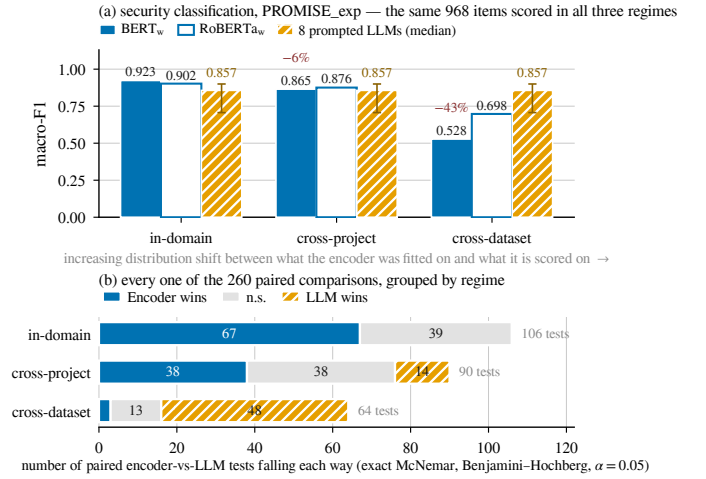


Fig. 3. (a) Security classification on PROMISE_exp; all three bars are scored on the same 968 items at every regime. The encoders are re-fitted at each regime; the third bar is the median of the eight prompted models, with the whisker spanning their range, and it is flat because those models are never fitted. The red annotations are BERT’s relative loss against its own in-domain score. (b) The 260 paired encoder-versus-LLM comparisons, grouped by the regime the encoder was evaluated under. Each unit on the axis is one exact McNemar test, not one requirement.

A. RQ1: within one corpus, supervision is still ahead

Reading the ID columns of Table VI, fine-tuned encoders hold the best score in five of the six cells. Class-weighted BERT leads on FR/NFR and on PROMISE_exp security, and class-weighted RoBERTa leads on all three sub-type granularities. The exception is SecReq security, where 4-bit Qwen2.5-7B posts the better point estimate (0.846 against 0.836), a lead that does not reach significance (exact McNemar $p = 0.696$; BH-adjusted 0.774).

Three observations qualify that ranking. Scale does not order the results: the 2.6 B Gemma-2-2B beats the hosted 70 B model on PROMISE_exp security, and the small open models fall well below the encoders on FR/NFR while staying competitive on security. The encoder advantage widens as the label set grows—from +0.014 on top-4 and +0.013 on top-6 to +0.095 on all-11 over the best prompted model—as one would expect if supervision mainly buys the long tail of rare classes. Cell sizes differ, so the top-4 ranking was recomputed on the 59 items every model answered; the best encoder still leads there by +0.036 (0.966 against 0.930), so the ordering is not an artefact of coverage. Every reportable model clears its majority-class baseline.

The paired tests make the in-domain verdict precise. Of the 106 in-domain encoder-versus-LLM comparisons, the encoder wins 67, 39 are inconclusive, and none is lost (Fig. 3 (b)).

B. RQ2: the ranking reverses under distribution shift

Table VII lists every measured transfer for the encoders, and Fig. 3 shows the comparison against the prompted models.

Cross-project transfer is survivable. Holding out unseen projects costs the encoders between 3% and 12% of their in-domain macro-F1 on PROMISE_exp, and up to 30% on

TABLE VI

MACRO-F1 FOR ALL TWELVE CONFIGURATIONS, ACROSS EVERY TASK, CORPUS AND EVALUATION REGIME. ID = IN-DOMAIN, XP = CROSS-PROJECT, XD = CROSS-DATASET. BEST PER COLUMN IN BOLD; “—” MARKS A CELL BELOW THE REPORTABILITY GATE OF SECTION III-F OR A REGIME THAT DOES NOT APPLY. A PROMPTED MODEL’S SCORE IS WRITTEN ONCE ACROSS THE REGIMES OF ITS CELL, BECAUSE IT CONSUMES NO TRAINING DATA AND THE REGIME CANNOT MOVE IT. ENCODER CELLS ARE CROSS-VALIDATED OVER ALL ITEMS ($n = 968$ FOR THE PROMISE_exp TASKS, 444 FOR SECReq, 353/433/524 FOR THE SUB-TYPE VARIANTS); PROMPTED CELLS ARE STRATIFIED SAMPLES OF THE SAME ITEMS ($n = 59-960$), AND ALL COMPARISONS ARE PAIRED ITEM-FOR-ITEM.

Configuration	Security						FR/NFR		NFR sub-types (PROMISE_exp)					
	PROMISE_exp			SecReq			PROMISE_exp		top-4		top-6		all-11	
	ID	XP	XD	ID	XP	XD	ID	XP	ID	XP	ID	XP	ID	XP
BERT _w	0.923	0.865	0.528	0.836	0.588	0.690	0.908	0.843	0.878	0.800	0.827	0.735	0.657	0.579
BERT _u	0.907	—	0.534	—	—	0.702	—	—	—	—	—	—	—	—
RoBERTa _w	0.902	0.876	0.698	0.824	0.668	0.696	0.896	0.853	0.936	0.835	0.837	0.753	0.724	0.688
RoBERTa _u	0.903	—	0.707	—	—	0.678	—	—	—	—	—	—	—	—
Gemini 3.1 Flash-Lite	0.829			0.713			0.867		0.922		—		—	
Llama-3.3-70B	0.891			0.814			0.852		0.907		0.782		—	
Qwen2.5-7B	0.884			0.846			0.656		0.876		0.824		0.576	
Llama-3.1-8B	0.872			0.845			0.682		0.700		0.717		0.629	
Gemma-2-2B	0.900			0.821			0.663		0.760		0.603		0.527	
Phi-4-mini	0.798			0.635			0.727		0.727		0.712		0.599	
SmolLM3-3B	0.841			0.717			0.562		0.763		0.693		0.482	
Qwen2.5-3B	0.707			0.741			0.554		0.732		0.689		0.543	
majority-class baseline	0.466			0.376			0.351		0.131		0.075		0.035	

the much smaller SecReq, whose three project groups make a held-out group a third of the corpus. The encoders remain ahead or tied in 76 of the 90 paired tests at this regime—the degradation NoBERT anticipated [4]: real, but moderate.

Cross-dataset transfer reverses the ranking. The class-weighted BERT that scores 0.923 on PROMISE_exp security falls to 0.528 on the same items once its training corpus is swapped for SecReq. That is a loss of 42.8% of its in-domain macro-F1, and it is the largest loss in the study; it belongs to the cross-dataset regime, and should not be read against the cross-project losses in the previous paragraph, whose maximum is 29.6%. RoBERTa is more robust ($0.902 \rightarrow 0.698$, a loss of 23%) and the reverse direction milder still (16–17%). The prompted models, having nothing to re-fit, move only with the difficulty of the corpus. Over the 64 cross-dataset comparisons the prompted models win 48 and the encoders 3; the in-domain tally of 67–39–0 becomes 3–13–48, on the same twelve models and the same scorer.

Corpus-level averages hide per-project failure. On the leave-one-project-out variant of FR/NFR, over the 37 projects with comparable coverage, every configuration—encoder and prompted alike—has at least one project it scores 0.000 on and one it scores 1.000 on. Only the middle of the distribution separates them: the encoders’ median per-project accuracy is 0.917 (s.d. 0.198) against medians of 0.545–0.750 (s.d. 0.213–0.301) for the small open models. A corpus-level average is a weak predictor of what a practitioner meets on one project.

The mechanism is a shift in class balance. A drop in score does not by itself say why a model failed, and the two candidate explanations—unfamiliar vocabulary and a shifted class prior—imply different remedies. The per-class evidence separates them (Fig. 4). After transfer from SecReq, BERT labels 44.2% of the PROMISE_exp items *security*, against a true prevalence of 12.9% in those items and 39.9% in the

TABLE VII

RELATIVE LOSS UNDER SHIFT FOR THE FOUR FINE-TUNED ENCODERS. $\Delta\%$ IS THE SHARE OF THAT CONFIGURATION’S OWN IN-DOMAIN MACRO-F1 LOST IN THE SHIFTED REGIME, ON THE SAME ITEMS. PROMPTED MODELS ARE ABSENT BECAUSE THEY HAVE NO IN-DOMAIN FIT TO LOSE; THEIR SCORES ARE IN TABLE VI. CROSS-DATASET APPLIES ONLY TO THE SECURITY TASK, THE ONE BOTH CORPORA ANNOTATE.

Fine-tuned encoder	In-dom.	Cross-project		Cross-dataset	
	macro-F1	macro-F1	$\Delta\%$	macro-F1	$\Delta\%$
<i>Security — PROMISE_exp</i>					
BERT _w	0.923	0.865	−6	0.528	−43
BERT _u	0.907	—	—	0.534	−41
RoBERTa _w	0.902	0.876	−3	0.698	−23
RoBERTa _u	0.903	—	—	0.707	−22
<i>Security — SecReq</i>					
BERT _w	0.836	0.588	−30	0.690	−17
RoBERTa _w	0.824	0.668	−19	0.696	−16
<i>FR/NFR</i>					
BERT _w	0.908	0.843	−7	—	—
RoBERTa _w	0.896	0.853	−5	—	—
<i>NFR sub-types, top-4</i>					
BERT _w	0.878	0.800	−9	—	—
RoBERTa _w	0.936	0.835	−11	—	—
<i>NFR sub-types, top-6</i>					
BERT _w	0.827	0.735	−11	—	—
RoBERTa _w	0.837	0.753	−10	—	—
<i>NFR sub-types, all-11</i>					
BERT _w	0.657	0.579	−12	—	—
RoBERTa _w	0.724	0.688	−5	—	—

corpus it was trained on. That 44.2% is a predicted class share, not a loss of score, and its closeness to the 42.8% above is a coincidence. Its consequence is visible per class: precision on the security class falls from 0.847 to 0.210 while recall falls far less (0.888 to 0.720). The model still finds the security requirements; it cannot stop finding them. That is the signature

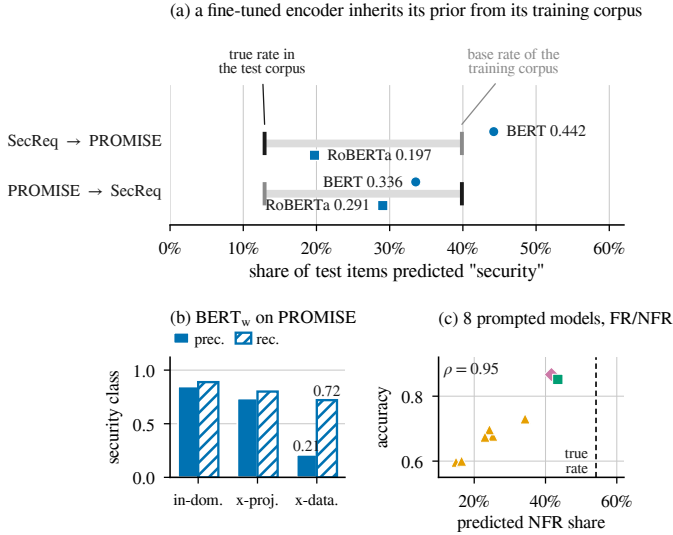


Fig. 4. One failure mode, two sources. (a) After cross-dataset transfer an encoder labels items at a rate pulled towards the base rate of the corpus it was trained on. The axis is the share of test items predicted *security*, not a score. (b) The consequence for the same model: precision on the security class collapses while recall is largely preserved. (c) The prompted models show the same defect from another source: on FR/NFR their accuracy ranks with how far their predicted class share sits from the true one.

of a shifted prior, not of a model that has failed to read the text.

The picture is not uniform, and we report it as measured. RoBERTa carries less of its source prior across (19.7% predicted against 12.9% true) and loses correspondingly less, and in the reverse direction both encoders predict *below* the target prevalence (33.6% and 29.1% against 39.9%), so content differences contribute there too. What the evidence supports is that the dominant term in the largest observed collapse is a prior the model brought with it. That is a comparatively encouraging diagnosis, since prior shift is correctable without labels from the target corpus [30], [31]. Applying such a correction would change the quantity being measured, so we leave it as the next step.

C. One failure mode, two sources

Section IV-B attributed the encoders' collapse to a prior carried over from training. Turning the same instrument on the prompted models finds the same defect there, arriving by a different route.

Prompted models also mis-set the class balance, and it explains most of their error. On FR/NFR the true NFR share is 54.2%, and every prompted model predicts NFR less often than that, from 14.9% (Qwen2.5-3B) to 43.4% (Llama-3.3-70B). Their accuracies rank almost exactly with that miscalibration (Spearman $\rho = 0.95$ over the eight reportable models, Fig. 4(c)), so the small open models' weak FR/NFR scores reflect a systematic bias towards answering FR rather than an inability to read the requirement.

The wording of the question moves that balance directly, which is what makes the link causal rather than correlational.

Across the nine paired cells of the phrasing sub-study (Table IV) the median spread between the three wordings is 0.044 macro-F1, but the largest is 0.485, for Gemma-2-2B on SecReq security. The predictions show what the *terse* wording does: it is the only variant phrased as a leading yes/no question, and under it Gemma-2-2B answers *security* for 96.7% of SecReq and 51.3% of PROMISE_exp items, against true rates of 39.7% and 12.7% in those frames. Qwen2.5-7B shifts the same way but less far. This is not a formatting artefact: the strict parser accepted 99.98% of the sub-study's 11,518 responses, so the answers were well-formed and simply wrong. Re-wording the question re-set the model's prior in the same way that re-training on another corpus re-set the encoder's. Prompt sensitivity of this kind is documented in the prompting literature [32], [33]; what is new here is that it lands on the same failure mode as the encoders' transfer loss.

Few-shot exemplars help where they carry information about the class balance. Across the 90 paired comparisons on the six local models—the only tier that ran few-shot—the median gain is +0.024 macro-F1, but the effect splits by task: slightly negative on the binary tasks (mean -0.007 over 54 comparisons; the 30 BH-significant effects average -0.027) and positive on the sub-types (mean $+0.062$ over 36; all 14 BH-significant effects are gains, averaging $+0.114$). Where the labels are a familiar convention the model already has a prior and the examples mostly perturb it, which is the over-prompting effect Tang et al. report [13]; where they are an unfamiliar eleven-class taxonomy, the exemplars are the only statement of what the classes mean.

These are one defect with two sources. The task needs a class prior that cannot be read off the requirement text, and each family acquires one from somewhere it should not: the encoder from its training corpus, the prompted model from its pre-training and from the wording of the question. Neither is calibrated to the corpus in front of it, which is why neither is safe to deploy on an in-distribution score alone.

D. RQ3: the accuracy–cost trade-off

Table V gives cost and latency by tier and Fig. 5 the frontier. Encoder inference costs \$0.0024 per 1,000 classifications on the imputed T4 rental. The prompted configurations range from \$0.0248 to \$0.0704 per 1,000 on the same basis, taking each model's median across its cells: 10 to 29 times the encoder, and about 19 times at the medians. One-off fine-tuning is small—a median of \$0.0093 per deployable model, about 277 GPU-seconds over all folds of a family—so the break-even of Eq. (1) is reached quickly: over all 276 encoder-versus-alternative pairings N^* lies between 46 and 552 items, with a median of 215. The Pareto frontier over reportable cells holds ten cells, nine of them encoders. The one prompted frontier point is 4-bit Qwen2.5-7B on SecReq security, which is the strongest freely available option on the one cell where transfer is what is being measured.

Two qualifications keep these figures in proportion. They are ratios measured on one machine at one date, and the prices behind them move: list rates change, and cost within

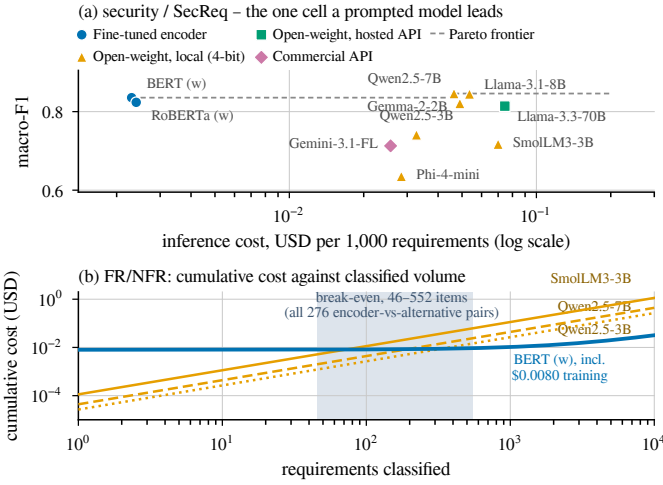


Fig. 5. RQ3. (a) SecReq security, the one cell a prompted model leads: 4-bit Qwen2.5-7B buys +0.011 macro-F1 for roughly 19 $\times$ the cost per item. (b) Cumulative cost against classified volume; fine-tuning is repaid within 46–552 classifications across all 276 pairings. Both panels use the cost basis of Section III-G.

the compute-priced tier scales with model size (Section III-G). And the model prices computation only—the human annotation an encoder needs before it can be fine-tuned is excluded, which favours the encoder.

Latency behaves differently from cost. The median per-request latency is 0.016s for the encoders, 0.095s for the hosted 70B model, 0.194–0.356s for the local open models and 0.614s for the commercial API. Free-tier queueing inflates the hosted *means* by up to two orders of magnitude, which is why all wall-clock projections use medians.

V. DISCUSSION

A. Generalizability

The most consequential result here is not that one family is better, but that the answer to “which is better?” changes between two protocols the literature treats as interchangeable, on the same corpora, the same items and the same scorer. A benchmark reporting only in-distribution numbers is not a weaker version of one that reports transfer; on this evidence it can report the opposite ordering. Two things follow. The regime belongs in the claim, next to the metric. And “generalisation” should mean held-out projects or an independently annotated corpus, not transfer across tasks within one dataset, which is what the term currently covers in parts of this literature [11].

The limits of our own claim should be equally explicit. We observe the reversal on one corpus pair, one task both corpora annotate, and one language. The study establishes that the reversal is possible and large when it happens, not that it occurs at any particular rate across requirements corpora. The mechanism we identify—a class prior carried over from the training corpus—is what would make it recur, and it can be checked on any new pair without re-running our models, by comparing the predicted class share against the target prevalence.

B. Consistency and reliability

Accuracy averaged over a corpus is not the only property a practitioner needs. Two kinds of instability appear in the results, one in each family.

The encoders are stable given a fixed training distribution and unstable when it moves: their scores change little across folds within a corpus but lose up to 42.8% of their in-domain macro-F1 when the training corpus is replaced. Their failure is at least predictable from something checkable before deployment, namely the class balance of the corpus they were fitted on relative to the target.

The prompted models are stable across regimes—by construction, since nothing is re-fitted—but unstable with respect to the question. Re-wording the same instruction, with the answer contract held fixed, moved macro-F1 by up to 0.485 in one cell. A deployment that pins its prompt gets repeatable answers, but the level at which they are repeatable was set by a wording choice no in-distribution study would surface. Both families also show wide per-project variance, with every configuration recording at least one project scored 0.000 and one scored 1.000. A corpus-level mean reported without a per-project spread overstates how reliable either family is in practice.

C. Accuracy–cost trade-off

For a team with a few hundred labelled requirements from the domain it will operate in, and any non-trivial volume, a fine-tuned encoder of about 110M parameters is both the more accurate and much the cheaper option, and it repays its training cost within a few hundred classifications. Entering an unlabelled or shifting domain changes the answer: a mid-size quantised open model gives up in-domain accuracy, degrades more gracefully, and runs on one consumer-class GPU without an API contract. What the results argue against is the implicit default of much recent work—fine-tuning on one corpus and assuming the result transfers.

The trade-off is a ratio, not a price. The absolute figures in Section IV-D are tied to one GPU, one date and two rate cards; what should survive is the order of magnitude between a 110M-parameter encoder and a multi-billion-parameter generative model doing the same classification, and the fact that within the compute-priced tier cost tracks model size. The annotation an encoder route needs is real and sits outside the model: a team without labels is not choosing between two prices per thousand items, it is choosing between annotating and not.

D. Contamination and availability

Both corpora are old, small and public, so prompted in-domain scores may be inflated by pre-training contamination [34]. That bias would overstate the prompted models in-domain, which is where we report the encoders winning, so the in-domain encoder advantage is conservative; and it cannot produce the cross-dataset reversal, since it would help the prompted models in both regimes equally. The pipeline, the frozen splits, the full prediction store and the scripts that regenerate every table and figure are public. Splits are hash-

and seed-pinned and scoring is a pure function of the stored raw outputs, so every number here reproduces without re-running a model.

VI. THREATS TO VALIDITY

Internal. The commercial tier ran at about 60 items per cell, where 23 of its 26 paired comparisons are inconclusive, so “never significantly beaten” partly reflects limited statistical power. Free-tier quotas caused missing-not-at-random dropout on some hosted configurations; the reportability gate excludes the affected cells rather than imputing them. Local models ran 4-bit quantised, which may understate full-precision accuracy, and each configuration used a single seed, so small encoder-to-encoder margins should not be over-read.

Construct. GPU costs are imputed from a published rental rate rather than billed, and token prices are 2026 list rates, so the ratios are more durable than the absolute values. N^* is a pure cost crossover computed regardless of the accuracy gap, and the cost model prices computation only, so the annotation an encoder needs is excluded—which favours the encoder. Strict scoring counts format non-compliance as error, matching deployment but differing from the lenient protocols of some prior work. The prompt is fixed by design (Section III-E); the phrasing sub-study bounds what a better wording would be worth on two models and nine cells, not on all eight.

External. Both corpora are small, dated and public, and their security label sets differ: PROMISE_exp’s is derived from the security sub-class and is non-functional by construction, whereas SecReq annotates any security-relevant sentence. The cross-dataset regime therefore carries definitional as well as distributional shift, on one corpus pair. Results cover English requirements and one NFR taxonomy.

Conclusion. All paired tests are exact and multiplicity-corrected as one family, and the frozen splits are stored once and never regenerated because grouped-fold membership is library-version dependent.

VII. CONCLUSION

We compared four fine-tuned encoder configurations against eight prompted LLMs on three requirements-classification tasks, holding the items, the label sets and the scoring rule fixed and varying only the distribution shift between what a model was fitted on and what it was scored on. Within a corpus the encoders are ahead and are never significantly beaten. Across an independently annotated corpus the ordering reverses and the prompted models are ahead in most comparisons, while the encoders remain an order of magnitude cheaper per item. The cause is the same in both families: a class prior acquired from somewhere other than the corpus being classified—from the training data in one case, from the wording of the instruction in the other.

An in-distribution score is therefore not a partial answer to the question of which approach to adopt; it can be the wrong one. Work in this area should report the evaluation regime as part of the result, and should treat the predicted class balance as a diagnostic to check before deployment rather than a detail of the confusion matrix.

REFERENCES

- [1] J. Cleland-Huang, R. Settimi, X. Zou, and P. Solc, “Automated classification of non-functional requirements,” *Requirements Eng.*, vol. 12, no. 2, pp. 103–120, 2007.
- [2] *ISO/IEC/IEEE 29148:2018(E) — Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, pp. 1–104, 30 Nov. 2018, doi: 10.1109/IEEESTD.2018.8559686.
- [3] Z. Kurtanović and W. Maalej, “Automatically classifying functional and non-functional requirements using supervised machine learning,” in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 490–495.
- [4] T. Hey, J. Keim, A. Koziolek, and W. F. Tichy, “NoRBERT: Transfer learning for requirements classification,” in *Proc. IEEE 28th Int. Requirements Eng. Conf. (RE)*, 2020, pp. 169–179.
- [5] F. Dalpiaz, D. Dell’Anna, F. B. Aydemir, and S. Çevikol, “Requirements classification with interpretable machine learning and dependency parsing,” in *Proc. IEEE 27th Int. Requirements Eng. Conf. (RE)*, 2019, pp. 142–152.
- [6] W. Alhoshan, A. Ferrari, and L. Zhao, “Zero-shot learning for requirements classification: An exploratory study,” *Inf. Softw. Technol.*, vol. 159, art. 107202, 2023.
- [7] A. El-Hajjaji, N. Fafin, and C. Salinesi, “Which AI technique is better to classify requirements? An experiment with SVM, LSTM, and ChatGPT,” arXiv:2311.11547, 2024.
- [8] K. Kaur and P. Kaur, “The application of AI techniques in requirements classification: A systematic mapping,” *Artif. Intell. Rev.*, vol. 57, art. 57, 2024, doi: 10.1007/s10462-023-10667-1.
- [9] S. N. Sonawane and S. M. Puthran, “Classification of functional and nonfunctional requirements based on convolutional neural network with flower pollination optimizer,” *Innov. Syst. Softw. Eng.*, 2024.
- [10] J. Peer, Y. Mordecai, and Y. Reich, “NLP4ReF: Requirements classification and forecasting—From model-based design to large language models,” in *Proc. IEEE Aerospace Conf.*, 2024, pp. 1–16.
- [11] M. Binkhonain and R. Alfayaz, “Are prompts all you need? Evaluating prompt-based large language models for software requirements classification,” *Requirements Eng.*, 2025.
- [12] W. Alhoshan, A. Ferrari, and L. Zhao, “How effective are generative large language models in performing requirements classification?” arXiv:2504.16768, 2025.
- [13] Y. Tang, D. Tuncel, C. Koerner, and T. Runkler, “The few-shot dilemma: Over-prompting large language models,” arXiv:2509.13196, 2025.
- [14] M. Chaudhary, C. Jain, and P. R. Anish, “Exploring zero-shot app review classification with ChatGPT: Challenges and potential,” arXiv:2505.04759, 2025.
- [15] P. Vasudevan, S. Reddivari, S. Ahuja, N. Niu, S. Kumar, B. Jung, and E. Gamess, “Requirements classification using fine-tuned transformer models: A comparative study of BERT, RoBERTa, and GPT-4o,” in *Proc. ACM Southeast Conf. (ACMSE)*, 2026, pp. 199–204.
- [16] O. Levy, I. Dikman, N. Levy, and M. Winokur, “AI-assisted requirements engineering: An empirical evaluation relative to expert judgment,” arXiv:2604.15222, 2026.
- [17] E. Knauss, S. Houmb, K. Schneider, S. Islam, and J. Jürjens, “Supporting requirements engineers in recognising security issues,” in *Proc. REFSQ*, 2011, pp. 4–18.
- [18] M. Lima, V. Valle, E. Costa, F. Lira, and B. Gadelha, “Software engineering repositories: Expanding the PROMISE database,” in *Proc. XXXIII Brazilian Symp. Softw. Eng. (SBES)*, 2019, pp. 427–436.
- [19] A. Ferrari, G. O. Spagnolo, and S. Gnesi, “PURE: A dataset of public requirements documents,” in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 502–505.
- [20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [21] Y. Liu *et al.*, “RoBERTa: A robustly optimized BERT pretraining approach,” arXiv:1907.11692, 2019.
- [22] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 10088–10115.
- [23] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [24] *ISO/IEC 25010:2011, Systems and Software Engineering—SQuaRE—System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.

- [25] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1993.
- [26] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [27] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [28] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.
- [29] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. R. Statist. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.
- [30] J. Quiñero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, Eds., *Dataset Shift in Machine Learning*. Cambridge, MA, USA: MIT Press, 2009.
- [31] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Comput.*, vol. 14, no. 1, pp. 21–41, 2002.
- [32] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proc. ICML*, 2021, pp. 12697–12706.
- [33] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," in *Proc. ACL*, 2022, pp. 8086–8098.
- [34] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. L. de Lacalle, and E. Agirre, "NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark," in *Findings of EMNLP*, 2023, pp. 10776–10787.